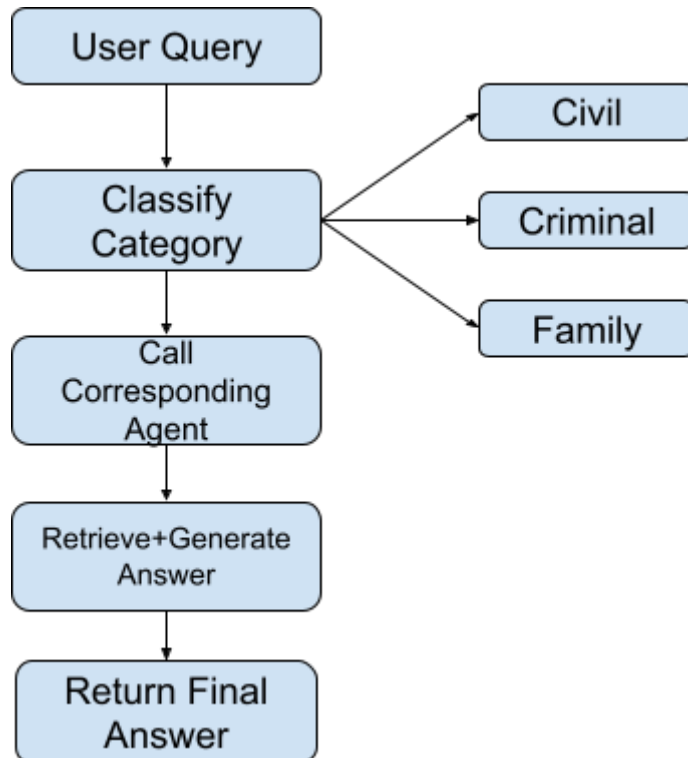


Multiagent Chatbot

Workflow



Import all required modules

- Google Gemini model and embeddings
- Vector DB ([Chroma](#))
- Text splitting from PDFs
- RetrievalQA (RAG pipeline)
- LangGraph (for agent orchestration)
- Typed states and graph utilities

```
[ ] !pip install langchain langgraph chromadb unstructured pypdf
    !pip install langchain-google-genai -U langchain-community langgraph
```

✓ Import Necessary libraries

```
import os
from langchain_core.tools import tool
from langgraph.graph import StateGraph
from langchain.chains import RetrievalQA
from langchain.vectorstores import Chroma
from langchain_core.runnables import Runnable
from langchain_core.messages import HumanMessage
from langchain.document_loaders import PyPDFLoader
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

Set Google API Key

- Sets **Google API key** to access **Gemini** models and embeddings.

```
os.environ["GOOGLE_API_KEY"] = "AIzaSyAuuWI7HyeoaryXbw2RdkbKZsFFjx0zawY"
```

Initialize LLM and Embedding Model

```
[ ] llm = ChatGoogleGenerativeAI(model="gemini-2.5-pro")
    embedding = GoogleGenerativeAIEmbeddings(model="models/embedding-001")
```

- **llm**: The language model used for generating responses.
- **embedding**: Generates vector embeddings for document chunks.

Load and Split PDF Documents

```
[ ] def process_pdf(pdf_path):
    loader = PyPDFLoader(pdf_path)
    docs = loader.load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
    return splitter.split_documents(docs)

civil_docs = process_pdf("Civil Laws.pdf")
criminal_docs = process_pdf("Criminal Laws.pdf")
family_docs = process_pdf("Family Laws.pdf")
```

process_pdf():

- Loads PDF using **PyPDFLoader**.
- Splits documents into chunks of 1000 characters with 100-character overlaps.

Prepares three separate document sets:

- **civil_docs**
- **criminal_docs**
- **family_docs**

Store Documents in Chroma Vector Store

```
[ ] civil_db = Chroma.from_documents(civil_docs, embedding, collection_name="civil")
criminal_db = Chroma.from_documents(criminal_docs, embedding, collection_name="criminal")
family_db = Chroma.from_documents(family_docs, embedding, collection_name="family")
```

- Stores each document chunk into **Chroma** using their respective embeddings.
- Enables **similarity-based retrieval** later during RAG.

Build Retrieval Agents

```
[ ] def build_retrieval_agent(db):  
    retriever = db.as_retriever()  
    return RetrievalQA.from_chain_type(llm=llm, retriever=retriever)  
  
civil_agent = build_retrieval_agent(civil_db)  
criminal_agent = build_retrieval_agent(criminal_db)  
family_agent = build_retrieval_agent(family_db)
```

- Wraps the Chroma vector DB with a retriever and `RetrievalQA` chain.
- Each agent fetches answers using Gemini and relevant document chunks.

Define Shared Graph State

```
[ ] from typing import TypedDict, Literal  
  
class ChatState(TypedDict):  
    query: str  
    category: Literal["civil", "criminal", "family"]  
    answer: str
```

`ChatState` defines the **shared state** between LangGraph nodes. It tracks:

- The user `query`
- Predicted `category`
- Final `answer`

Category Classifier

```
def classify_category(state: ChatState):
    query = state["query"]

    prompt = f"""
You are a legal assistant.
Classify the following query into one of these categories: 'civil', 'criminal', or 'family'.
Query: "{query}"
Only respond with one word: civil, criminal, or family.
"""

    response = llm.invoke(prompt)
    category = response.content.strip().lower()
    if category not in ["civil", "criminal", "family"]:
        category = "civil"
    return {"category": category}
```

- Extracts the user query from `state["query"]`.
- Sends the query to an LLM with a prompt to classify it as **"civil"**, **"criminal"**, or **"family"**.
- Ensures the LLM responds with only one word.
- Converts the response to lowercase and trims whitespace.
- If the response is not one of the three valid categories, it defaults to **"civil"**.
- Returns the result as a dictionary: `{"category": category}`.

Agent Functions

```
[ ] def call_civil_agent(state: ChatState):
    answer = civil_agent.run(state["query"])
    return {"answer": answer}

def call_criminal_agent(state: ChatState):
    answer = criminal_agent.run(state["query"])
    return {"answer": answer}

def call_family_agent(state: ChatState):
    answer = family_agent.run(state["query"])
    return {"answer": answer}
```

- Each function takes the query, runs it through the corresponding agent, and returns the result.

Define LangGraph Workflow

```
[ ] graph = StateGraph(ChatState)

graph.add_node("classify", classify_category)
graph.add_node("civil_agent", call_civil_agent)
graph.add_node("criminal_agent", call_criminal_agent)
graph.add_node("family_agent", call_family_agent)

graph.set_entry_point("classify")
```

- Defines a **state machine** where each node represents a function.
- Entry point: `classify` → this node decides which category the query falls into.

Conditional Routing

```
[ ] graph.add_conditional_edges("classify", lambda state: state["category"], {
    "civil": "civil_agent",
    "criminal": "criminal_agent",
    "family": "family_agent"
})

graph.set_finish_point("civil_agent")
graph.set_finish_point("criminal_agent")
graph.set_finish_point("family_agent")
```

- Based on the classification result, the graph routes to the corresponding agent node.
- Each of those agent nodes is a terminal node (finishes the graph).

Compile and Run the Graph

```
[ ] chat_app = graph.compile()
```

- Converts the designed graph into an executable application using LangGraph.

Run a Sample Query

```
[ ] query = "What are the consumer protection laws?"

result = chat_app.invoke({"query": query})
print("\n Answer:", result["answer"])
```

Executes the graph with a sample query.

The process flow is:

1. Classify → Civil
2. Run `civil_agent` → Get answer
3. Print the answer

Output in Swagger

POST /ask Ask Legal Bot

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "query": "Explain child adoption"
}
```

Responses

Curl

```
curl -X 'POST' \
  'https://c2e23a4ba5a7.ngrok-free.app/ask' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "query": "Explain child adoption"
  }'
```

Request URL

```
https://c2e23a4ba5a7.ngrok-free.app/ask
```

Server response

Code	Details
200	Response body <pre>{ "answer": "Based on the context provided, here is an explanation of child adoption in India:\n\nAdoption is a significant aspect of Indian family law. The legal framework for adoption varies depending on one's religion.\n\n**For Hindus, Buddhists, Jains, and Sikhs:** The **Hindu Adoption and Maintenance Act, 1956** governs adoption. This law permits both men and women to adopt, subject to certain conditions regarding age and marital status.\n\n**For Muslims, Christians, and Parsis:** Traditionally, the personal laws of these religions do not recognize adoption. However, they can adopt a child under the **Juvenile Justice (Care and Protection of Children) Act, 2015**. This secular law provides a uniform procedure for adoption that is applicable to all citizens, irrespective of their religion." }</pre> Download Response headers <pre>content-length: 812 content-type: application/json date: Fri, 25 Jul 2025 06:58:50 GMT ngrok-agent-ips: 34.81.75.142 server: uvicorn</pre>